AI PERSONAL STUDY ASSISTANT

Study Material: Python Programming Basics

Submitted by	Nakshathra joy
Department	computer engineering
College	TKMIT
Course	cs(aiml)
Academic Year	2026–2030

1. Introduction to Python

What is Python? Python is a high-level, general-purpose programming language. Its syntax is designed to be readable, which makes it a useful first language for students. A Python interpreter runs the instructions written in a program.

Why is Python important? Python helps students learn computational thinking: breaking a problem into smaller steps, writing instructions, testing results, and correcting mistakes.

Key features

- **Readable syntax:** many instructions look close to everyday language.
- **Interpreted:** programs can be run without manually compiling each source file in the usual beginner workflow.
- **Large standard library:** built-in modules help with common tasks.
- **Cross-platform:** Python is available on major operating systems.
- **Multi-purpose:** it supports scripting, applications, data analysis, and AI-related work.

Applications of Python

Area	Example use
Education	Learning programming and solving logic problems
Automation	Renaming files or processing repeated tasks
Data science	Cleaning data and producing charts
AI and machine learning	Building prototypes that use models and datasets
Web development	Creating server-side logic for websites

First Python program

```
print("Hello, world!")
```

Output: **Hello, world!** The `print()` function displays information. Strings (text) are usually enclosed in single or double quotation marks.

Learning tip: type each example yourself, predict the output, run the code, and explain why the output appears. This builds understanding more effectively than copying code without testing it.

1. Variables and Data Types

A **variable** is a name that refers to a value. In Python, use the assignment operator = to assign a value. Variable names should be meaningful and cannot begin with a digit.

```
student_name = "Asha" age = 18
```

```
percentage = 96.5 is_enrolled = True
```

Dynamic typing means Python determines the type of a value while the program runs. You do not need to write the type when creating a variable. A variable can later refer to a value of another type, although doing so should be clear and intentional.

```
value = 10# currently an integer value = "ten"# now a string
```

Common data types

Type	Meaning	Example
int	Whole numbers	count = 12
float	Numbers with a decimal part	price = 19.50
str	Text	city = "Kochi"
bool	Logical value: True or False	is_ready = False

Check a value's type with `type()`:

```
score = 96 print(type(score))
```

This displays the integer type. Note that `True` and `False` are written with capital first letters in Python.

Naming tips

- Use descriptive names such as `total_marks` instead of `x` when the meaning matters.
- Python is case-sensitive: `score` and `Score` are different names.
- Avoid using Python keywords such as `if` or `while` as variable names.

1. Operators and Expressions

An **operator** performs an action on values. An **expression** combines values, variables, and operators to produce a result.

Arithmetic operators

Operator	Purpose	Example and result
+	Addition	7 + 2 □ 9
-	Subtraction	7 - 2 □ 5
*	Multiplication	7 * 2 □ 14
/	Division	7 / 2 □ 3.5
//	Floor division	7 // 2 □ 3
%	Remainder	7 % 2 □ 1
**	Exponentiation	2 ** 3 □ 8

Comparison and logical operators

Kind	Operators	Example
Comparison	==, !=, >, <, >=, <=	5 == 5 gives True
Logical	and, or, not	True and False gives False

The assignment operator `=` stores a value; the comparison operator `==` checks equality. They are not interchangeable.

Operator precedence

Python follows a priority order. Parentheses are evaluated first, then exponentiation, then multiplication/division/floor division/remainder, then addition/subtraction, followed by comparisons and logical operations. When unsure, use parentheses to make the intended order clear.

```
result = (50 - 5 * 6) // 4

print(result)  # 5
```

Step-by-step: 5 × 6 = 30; 50 □ 30 = 20; 20 // 4 = 5.

Example of logical operators

```
age = 18 has_id = True

print(age >= 18 and has_id)  # True
```

The expression is true only when both conditions are true. The `or` operator is true if at least one condition is true; `not` reverses a Boolean value.

1. Conditional Statements and Loops

Conditional statements let a program choose between actions based on a condition. Indentation defines which statements belong to each block.

if and else

```
marks = 72

if marks >= 50: print("Pass")

else:

print("Try again")
```

If the condition is true, the first block runs; otherwise, the else block runs.

if, elif and else

```
score = 85

if score >= 90: grade = "A"

elif score >= 75: grade = "B"

else:
```

```
grade = "C" print(grade)  # B
```

`elif` means “else if” and allows another condition to be tested when earlier conditions are false.

for loop

A for loop repeats over items in a sequence. The `range()` function can generate a sequence of numbers.

```
for number in range(1, 4): print(number)

# Output: 1, 2, 3
```

while loop

A while loop repeats while its condition remains true. Update the relevant variable so the loop can finish.

```
count = 1

while count <= 3: print(count) count += 1
```

Loop safety and good practice

- A loop can repeat forever if its condition never becomes false; this is called an infinite loop.
- Check the starting value, condition, and update step.
- Use a for loop when iterating through a sequence or a known range; use a while loop when repetition depends on a condition.
- Indent each block consistently, usually with four spaces.

1. Functions

A **function** is a reusable block of code that performs a task. Functions help organize a program, reduce repeated instructions, and make code easier to test.

Defining and calling a function

```
def greet():

                                        print("Welcome to Python!")



greet()
```

The keyword `def` defines the function. The indented lines form its body. The function runs when it is called by name with parentheses.

Parameters and arguments

A **parameter** is a name in the function definition. An **argument** is the value passed when the function is called.

```
def greet_student(name): print("Hello,", name)



                                    greet_student("Asha")  # Hello, Asha
```

Return statement

The `return` statement sends a result back to the code that called the function. This differs from simply printing a result.

```
def add(a, b): return a + b
```

```
answer = add(3, 4) print(answer)  # 7
```

In this example, the function adds two values and returns 7. The returned value is stored in `answer` and then displayed.

Why functions are useful

- **Reusability:** call the same function with different inputs.
- **Clarity:** give each function a clear name and one main purpose.
- **Maintenance:** change a task in one place rather than in many repeated blocks.
- **Testing:** check a function using different inputs and expected outputs.

Good function design tip: choose meaningful names, pass required information as parameters, and return a result when other code needs to use it.

1. Practice Questions

Try these questions before checking the answer key. They cover the core concepts introduced in this document.

Multiple-choice questions

1. **Which function displays text on the screen?**

A. input() B. print() C. type() D. range()

1. **What is the data type of 3.14?**

A. int B. str C. float D. bool

1. **What is the result of 17 // 5?**

A. 2 B. 3 C. 3.4 D. 5

1. **Which operator checks whether two values are equal?**

A. = B. != C. == D. **

1. **Which keyword defines a function?**

A. loop B. def C. func D. return

1. **What does elif allow a program to do?**

A. Repeat forever B. Define a variable C. Test another condition D. End Python

Short-answer and programming questions

1. Explain dynamic typing in Python in one or two sentences.
2. State one difference between a for loop and a while loop.
3. Write a program that prints whether a number is even or odd.
4. Write a function named square that takes a number and returns its square.

1. Answers, Key Takeaways and Conclusion

Answer key

1. **B — print().** It displays text or values.
2. **C — float.** The value 3.14 has a decimal part.
3. **B — 3.** Floor division returns the quotient rounded down; 17 divided by 5 is 3 remainder 2.
4. **C — ==.** The single equals sign assigns a value.
5. **B — def.** It begins a function definition.
6. **C — Test another condition.** elif is checked if earlier conditions are false.
7. Python determines a value's type at runtime, so the type does not need to be declared in advance.
8. A for loop commonly iterates over a sequence or range; a while loop repeats while a condition is true.
9. Example:

```
number = 7
```

```
if number % 2 == 0: print("Even")

else:

print("Odd")
```

1. Example:

```
def square(number):

return number * number



print(square(4))  # 16
```

Key takeaways

- Python is a readable, versatile language used across many fields.
- Variables refer to values; common types include int, float, str, and bool.
- Operators perform calculations, comparisons, and logical checks.
- Conditions control choices, loops repeat work, and functions package reusable tasks.
- Testing code and explaining the output are important parts of learning.

Conclusion

Python provides a strong foundation for programming and later study in data science, artificial intelligence, and machine learning. By practising the examples and solving the questions, students can strengthen their problem-solving skills. In the AI Personal Study Assistant project, this PDF can serve as a knowledge source for a chatbot that answers student questions about the topics covered here.

Document note: Replace all title-page placeholders with your correct details before submission. Review formatting and page count after editing.